TEORÍA DE COMPILADORES — TRABAJO DE CURSO

ELABORADO CON LA ASISTENCIA DE IA Y CORREGIDO Y REVISADO POR CARLOS ANDRÉS ARBELÁEZ VELÁSQUEZ

Entrega 2

Análisis Sintáctico

Descendente Recursivo y Predictivo Descendente

Construcción de un analizador sintáctico sobre la gramática BNF definida en la Entrega 1, implementando dos métodos: descendente recursivo (con visualización del árbol de análisis) y predictivo descendente (con tabla LL y proceso de pila), más presentación oral del proyecto.

MODALIDAD	ENTREGA	PRESENTACIÓN	DURACIÓN ORAL
Equipos de trabajo	**2**	**Jueves 24 de septiembre, 2026**	**10 minutos por equipo**

Continuación del proyecto iniciado en la Entrega 1 — construye sobre el analizador léxico ya desarrollado

§ 1 — CONTEXTO Y CONTINUIDAD

Descripción general de la Entrega 2

Esta segunda entrega se construye directamente sobre los productos de la Entrega 1: la Gramática Libre de Contexto (GLC) en formato BNF y el analizador léxico ya implementado. El objetivo de esta entrega es avanzar en la fase de análisis sintáctico del compilador, implementando dos métodos distintos de análisis descendente sobre la misma gramática ya diseñada.

El equipo deberá diseñar e implementar una única aplicación que integre ambos métodos de análisis, permita seleccionar o ingresar cadenas de entrada, y visualice gráficamente los resultados de cada método. La aplicación debe reutilizar el módulo lexer de la Entrega 1 como etapa previa de tokenización.

REQUISITO PREVIO
Para realizar esta entrega, la gramática propuesta en la Entrega 1 debe cumplir las condiciones LL: sin recursividad por la izquierda y con conjuntos PRIMERO disjuntos para cada no-terminal. Si durante el desarrollo de esta entrega el equipo detecta que su gramática no cumple alguna de estas condiciones, debe corregirla y documentar los cambios realizados.

§ 2 — MÉTODOS DE ANÁLISIS REQUERIDOS

Los dos métodos de análisis sintáctico

La aplicación debe implementar y ofrecer al usuario los dos siguientes métodos de análisis sintáctico descendente. Ambos operan sobre el flujo de tokens producido por el analizador léxico de la Entrega 1.

Método 1: Análisis Sintáctico Descendente Recursivo

El análisis descendente recursivo implementa directamente la gramática mediante un conjunto de funciones mutuamente recursivas: una función por cada no-terminal de la gramática. Cada función intenta reconocer la producción correspondiente consumiendo tokens del flujo de entrada.

Requerimientos del Método 1

1. **Implementación del parser recursivo:** Cada no-terminal de la gramática BNF entregada en la Entrega 1 debe tener su función correspondiente (por ejemplo, parse_sent_si(), parse_expresion(), etc.). Las funciones deben consumir tokens usando el lexer de la Entrega 1.
2. **Construcción del árbol de análisis sintáctico:** El parser debe construir explícitamente el árbol de análisis sintáctico (parse tree) a medida que avanza el reconocimiento. Cada

nodo interior del árbol corresponde a un no-terminal y cada hoja a un terminal (token) reconocido.

3. **Visualización gráfica del árbol:** El árbol de análisis sintáctico debe mostrarse **gráficamente** al usuario. Se recomienda el uso de bibliotecas como graphviz, matplotlib + networkx, anytree + rich, o una visualización en consola mediante arte ASCII/Unicode. El árbol debe ser legible, mostrar claramente la jerarquía de nodos y distinguir visualmente los no-terminales de los terminales.
4. **Resultado del análisis:** Al finalizar, el programa debe indicar claramente si la cadena de entrada es válida o inválida según la gramática. No es obligatorio recuperarse de errores más allá del primero encontrado: basta con reportar el primer error sintáctico es suficiente.

NOTA SOBRE LA CONSTRUCCIÓN DEL ÁRBOL EN EL MÉTODO RECURSIVO
El árbol de análisis sintáctico puede construirse de dos formas equivalentes: (a) cada función de parse retorna un nodo que ya contiene sus hijos construidos por llamadas recursivas, o (b) se pasa un nodo padre como parámetro y cada función agrega sus hijos directamente. Cualquiera de los dos enfoques es válido siempre que el árbol resultante represente fielmente la derivación realizada.

Método 2: Análisis Sintáctico Predictivo Descendente (Tabla LL)

El análisis predictivo descendente es un método tabulado que utiliza una tabla de análisis sintáctico LL(x) y una pila explícita para guiar el reconocimiento de la cadena de entrada. La tabla codifica, para cada par (no-terminal, token de la entrada), qué producción de la gramática debe aplicarse.

Requerimientos del Método 2

5. **Cálculo y presentación de la tabla de análisis LL(x):** El programa debe contener la tabla de análisis sintáctico LL(x) y gráficamente al usuario antes o durante el análisis, con filas indexadas por no-terminal y columnas por terminal (incluyendo $). Las celdas contienen la producción a aplicar o un indicador de error.
6. **Proceso de análisis con pila explícita:** El análisis debe realizarse mediante el algoritmo clásico de pila: se inicializa la pila con el símbolo inicial y $; en cada paso se consulta la tabla según el tope de la pila y el token actual de la entrada; se expande o se consume según corresponda. El programa debe mostrar gráficamente, paso a paso, el estado de la pila, el token actual de la entrada y la entrada en la tabla que se utilizó en ese paso.
7. **Visualización del proceso:** La traza del análisis debe presentarse como una tabla de pasos con al menos las siguientes columnas: número de paso, contenido de la pila, token actual (lookahead), producción o acción tomada. Se valorará resaltar la celda de la tabla LL(x) consultada en cada paso.
8. **Construcción y visualización del árbol de análisis sintáctico:** Al igual que en el Método 1, el proceso de análisis predictivo debe construir y mostrar gráficamente el árbol de análisis sintáctico resultante. El árbol debe reflejar fielmente las expansiones realizadas durante el algoritmo de pila.
9. **Resultado del análisis:** Al finalizar, el programa debe indicar si la cadena es válida o inválida. No es obligatorio identificar varios errores: con reportar el primer error sintáctico es suficiente.

TABLA LL(X) — ESTRUCTURA ESPERADA

La tabla LL(x) es una matriz M[A, a] donde A es un no-terminal y a es un terminal o $.

M[A, a] = A -> alpha si a esta en PRIMERO(alpha), o si epsilon esta en PRIMERO(alpha) y a esta en SIGUIENTE(A).

M[A, a] = error en cualquier otro caso.

El programa debe calcular esta tabla a partir de la gramatica de la Entrega 1 (no ingresarla manualmente).

§ 3 — REQUERIMIENTOS DE LA APLICACIÓN

Requerimientos funcionales de la aplicación

Independientemente del método de análisis elegido, la aplicación debe cumplir los siguientes requerimientos generales de interfaz e integración:

10. **Selección de cadena de entrada:** El usuario debe poder elegir entre: (a) ingresar una nueva cadena de texto libre, o (b) seleccionar una cadena de una lista de cadenas predefinidas que cubran variedad de constructos del lenguaje (condicionales, ciclos, funciones, expresiones aritméticas, etc.). Estas cadenas predefinidas pueden ser las mismas de la Entrega 1 u otras nuevas.
11. **Selección del método de análisis:** El usuario debe poder elegir entre el Método 1 (descendente recursivo) y el Método 2 (predictivo descendente LL) antes de ejecutar el análisis.
12. **Integración con el lexer de la Entrega 1:** La etapa de tokenización debe realizarse mediante el módulo lexer.py ya desarrollado, sin duplicar su lógica. El parser recibe el flujo de tokens producido por el lexer.
13. **Modularidad del código:** El código debe organizarse en módulos separados: uno para el parser recursivo, otro para el parser predictivo, otro para la construcción de la tabla LL, y otro para la visualización. El módulo principal orquesta la interacción con el usuario.
14. **Portabilidad:** La aplicación debe ejecutarse en Python 3.9 o superior. Todas las dependencias externas (bibliotecas para gráficos, etc.) deben listarse en un archivo requirements.txt incluido en la entrega.

§ 4 — PRESENTACIÓN ORAL

Presentación del proyecto ante el grupo

Además de la entrega del código y la documentación, cada equipo debe preparar y presentar su proyecto ante el grupo en la fecha y condiciones indicadas a continuación.

FECHA Y HORA DE PRESENTACIÓN

Fecha: Jueves 24 de septiembre de 2026

Horario: En el horario habitual de clase

Duración: 10 minutos por equipo (demostración en vivo)

Orden: Se anunciará al inicio de la sesión según orden de entrega en Moodle

Estructura sugerida de la presentación (10 minutos)

Tiempo	Contenido	Responsable
0–2 min	Recapitulación: lenguaje diseñado, gramática BNF y lexer de la Entrega 1.	Equipo
2–5 min	Demostración en vivo del Método 1 (descendente recursivo): ingresar una cadena, mostrar el árbol de análisis sintáctico resultante y una cadena inválida con su reporte de error.	Equipo
5–8 min	Demostración en vivo del Método 2 (predictivo LL): mostrar la tabla LL(1), ejecutar el análisis paso a paso con la pila visible, y mostrar el árbol resultante.	Equipo
8–10 min	Conclusiones: diferencias observadas entre los dos métodos, dificultades encontradas, decisiones de diseño tomadas.	Equipo

Aspectos que se evaluarán en la presentación

- Claridad al explicar el funcionamiento de cada método de análisis.
- Correcto funcionamiento de la aplicación durante la demostración en vivo.
- Capacidad de responder preguntas sobre las decisiones de implementación tomadas.
- Comprensión de los conceptos teóricos: PRIMERO, SIGUIENTE, tabla LL, árbol sintáctico.
- Demostración de una cadena inválida con el reporte de error correspondiente.

RECOMENDACIÓN

Ensayen la demostración en vivo antes del día de la presentación. Tengan preparadas al menos dos cadenas válidas y una inválida que demuestren diferentes constructos del lenguaje. Asegúrense de que la aplicación funcione correctamente en el equipo desde el que van a presentar.

§ 7 — ENTREGABLES

Resumen de entregables

La entrega debe realizarse en la fecha acordada e incluir los siguientes elementos:

E1 Código fuente
Todos los módulos del proyecto (lexer reutilizado + nuevos módulos de la Entrega 2), organizados según la estructura sugerida en §3.

E2 Tabla LL(1) calculada
Documento o sección que muestre los conjuntos PRIMERO y SIGUIENTE calculados para cada no-terminal, y la tabla LL(1) completa de la gramática.

E3 README actualizado
Instrucciones claras de instalación de dependencias (requirements.txt) y ejecución de la aplicación, incluyendo cómo cambiar entre los dos métodos de análisis.

E4 Presentación oral
Presentación en vivo el jueves 9 de abril de 2026, demostrando el funcionamiento de ambos métodos de análisis sobre cadenas representativas del lenguaje.

E5 Ajustes a la gramática (si aplica)
Si fue necesario modificar la gramática de la Entrega 1 para cumplir la condición LL(1), documentar los cambios realizados y su justificación.

E6 Capturas de pantalla / ejemplos
Al menos 3 capturas de pantalla o salidas de consola que muestren: el árbol sintáctico del Método 1, la traza de la pila del Método 2, y un caso de error.

§ 8 — EVALUACIÓN

Criterios de evaluación

Criterio	Peso	Descripción
Método 1 — Parser recursivo funcional	**10%**	Reconoce correctamente cadenas válidas e inválidas de la gramática.
Método 1 — Árbol sintáctico (recursivo)	**10%**	El árbol se construye y visualiza correctamente, reflejando la derivación.
Método 2 — Tabla LL(1) correcta	**10%**	Conjuntos PRIMERO y SIGUIENTE correctos; tabla LL(1) completa y visible.
Método 2 — Proceso de pila visible	**10%**	Traza paso a paso con pila, lookahead y producción/acción aplicada.
Método 2 — Árbol sintáctico (predictivo)	**10%**	El árbol se construye y visualiza correctamente para el método predictivo.
Presentación oral	**40%**	Claridad, demostración en vivo correcta, respuesta a preguntas.
Calidad del código y documentación	**10%**	Código modular, legible, documentado; README y requirements.txt presentes.

Teoría de Compiladores — Entrega 2 — Análisis Sintáctico Descendente Recursivo y Predictivo

Este documento establece los requisitos mínimos. El equipo puede ir más allá de lo requerido siempre que cumpla con todos los criterios indicados. Las entregas futuras avanzarán hacia el análisis semántico y la generación de código.